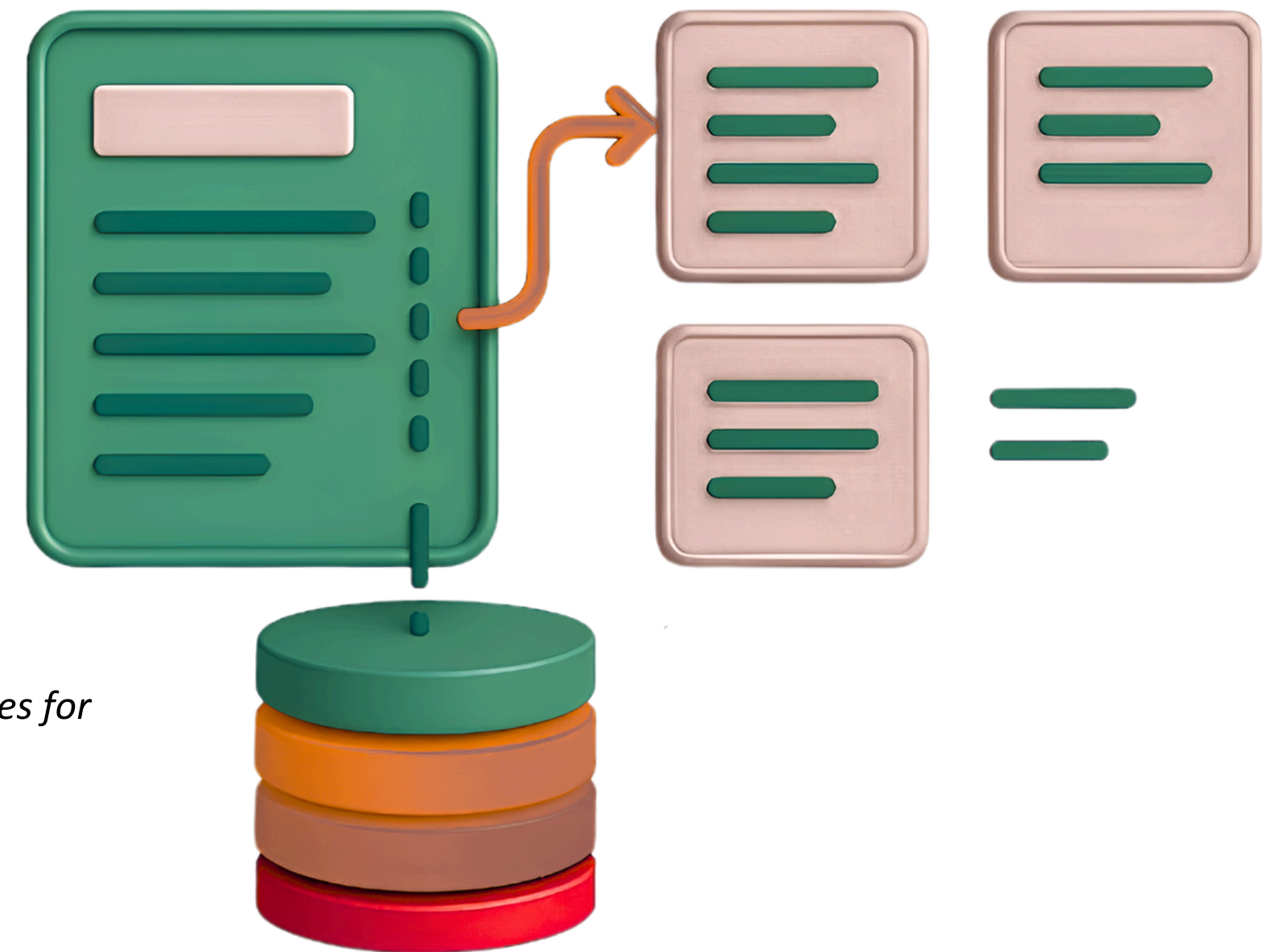


Retrieval Strategies & Chunking

Module 8 · Session 2 · 3 Hours

The piece Session 1 glossed over: how documents actually get split, and the real strategies for finding the right pieces again.



Session Roadmap

Chunking fundamentals and advanced techniques, then real alternative retrieval strategies — a lab comparing them, and a discussion on trade-offs.

0:00	●	Chunking Fundamentals — why it matters, fixed vs semantic, overlap	30 min
0:30	●	Advanced Chunking — recursive splitting, metadata enrichment	30 min
1:00	●	Retrieval Strategies — dense, sparse (BM25), multi-query, parent-child	30 min
1:30	●	Break	15 min
1:45	●	Hands-on Lab — build and compare chunking + retrieval strategies	45 min
2:30	●	Discussion — chunk size vs context vs precision trade-offs	30 min

New today: BM25 (rank_bm25) — a real, pure-Python keyword-based retrieval algorithm, no network or API needed at all.

The Piece We Glossed Over

Session 1 gave you

A complete, real RAG pipeline — but the documents were pre-cut, short sentences, chunked for you without a second thought

The gap

Real documents are long, messy, and structured — HOW you cut them into chunks quietly determines everything retrieval can find later

Today

Chunking done deliberately, plus real alternatives to pure dense search — because vector similarity alone has real, known blind spots

Slide 23's field guide from Session 1 said it plainly: retrieval quality caps generation quality. Today is where retrieval quality actually gets built, not assumed.

What You'll Be Able to Do

1

Understand chunking principles: why it matters, fixed-size vs semantic chunking, overlap strategies

2

Apply advanced chunking techniques: recursive splitting, sentence-level chunking, metadata enrichment

3

Implement multiple retrieval strategies: dense, sparse (BM25), multi-query, and parent-child retrieval

4

Build and evaluate real chunking and retrieval pipelines, and reason about the trade-offs between them

1

SECTION

Chunking Fundamentals

Why HOW you split a document quietly decides what retrieval can ever find



Session 16 of 40

THE PROBLEM, MADE CONCRETE

Why Chunking Matters

Chunk: a piece of a document small enough to embed and retrieve precisely, but large enough to still make sense on its own.

Chunks too large

- One embedding represents a whole page — averages away the specific detail a query needs
- Retrieval returns mostly-irrelevant text alongside the one relevant sentence
- Wastes context window tokens (M2-S2 cost callback) on padding

Chunks too small

- A single sentence may lose the surrounding context that gives it meaning
- "It costs \$50" retrieved alone — \$50 for WHAT?
- More chunks to store and search — real cost and latency at scale

TWO PHILOSOPHIES

Fixed-Size vs. Semantic Chunking

Approach	How it works	Trade-off
Fixed-size	Cut every N characters (or tokens), regardless of content	Simple, predictable, fast — today's lab default
Semantic	Cut at natural meaning boundaries — paragraph, topic, or section breaks	Chunks stay coherent, but sizes vary unpredictably

In other words: fixed-size is cutting a rope every meter regardless of what's tied to it; semantic is cutting at the knots — messier lengths, but nothing important gets severed mid-idea.

Session 2's lab uses fixed-size (via recursive splitting, Slide 10) because it's the practical default most real pipelines start with — semantic chunking is a genuine upgrade path, not a requirement.

TWO MORE REAL DECISIONS

Overlap Strategies & Document-Type Considerations

Overlap: letting adjacent chunks share some text at their boundary — so an idea split across a chunk boundary still appears whole in at least one chunk.

Overlap size

Typically 10-20% of chunk size — enough to catch boundary-split ideas, not so much you're storing near-duplicates

Prose documents

Recursive splitting (paragraph → sentence → word) works well — natural language has natural break points

Structured documents (code, tables, JSON)

Fixed-size or naive splitting can break syntax mid-structure — these need format-aware splitting, a real, harder problem

2

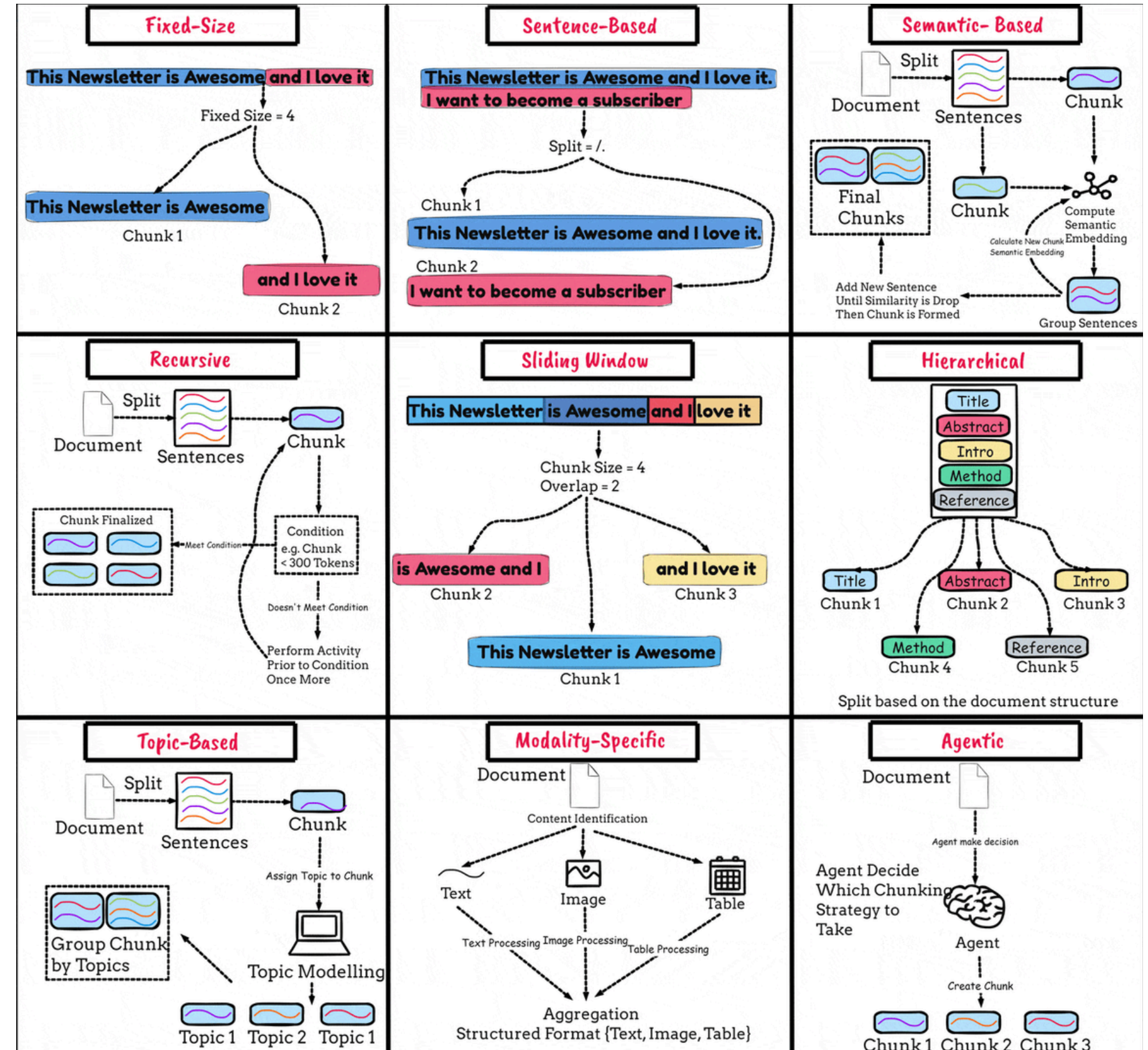
SECTION

Advanced Chunking

Real, working splitters — and the metadata that makes chunks traceable



Session 16 of 40



THE PRACTICAL DEFAULT

Recursive Character Splitting

Worked example — real LangChain, from Module 8·S1

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=500,      # target characters per chunk
    chunk_overlap=50,    # shared context between adjacent chunks
)
chunks = splitter.split_text(document_text)
# Tries paragraph breaks first, then sentences, then words --
# only splits mid-word as an absolute last resort
```

Why "recursive"

- It tries the BEST separator first (paragraph breaks), then falls back to sentences, then words
- Only splits mid-word as an absolute last resort — this is what makes fixed-size chunking feel almost semantic in practice
- The exact splitter this course already installed in M8·S1 — no new dependency today

A DIFFERENT BOUNDARY RULE

Sentence & Paragraph-Level Chunking

Sentence-level

Each chunk is exactly one (or a few) complete sentences — maximizes precision, at the cost of losing surrounding context entirely

Paragraph-level

Each chunk is one complete paragraph — a natural middle ground for well-structured prose documents

The real trade-off vs. recursive splitting

Recursive splitting optimizes for a TARGET SIZE and uses sentence/paragraph breaks as a preference; sentence/paragraph chunking optimizes for BOUNDARY PURITY and lets size vary freely

DON'T JUST STORE TEXT

Metadata Enrichment

Metadata enrichment: attaching structured facts to each chunk beyond its raw text — source document, position, section title — usable for filtering (M8-S1) and citation.

source

Which original document this chunk came from — required for real citation

chunk_index

This chunk's position in its source — lets you reconstruct order, or fetch neighboring chunks

section / heading

What part of the document this is ("Returns Policy", "Chapter 3") — a real, structured filter beyond pure semantics

char_count / token_count

Useful for debugging retrieval quality later — is a bad answer coming from a too-small chunk?

3

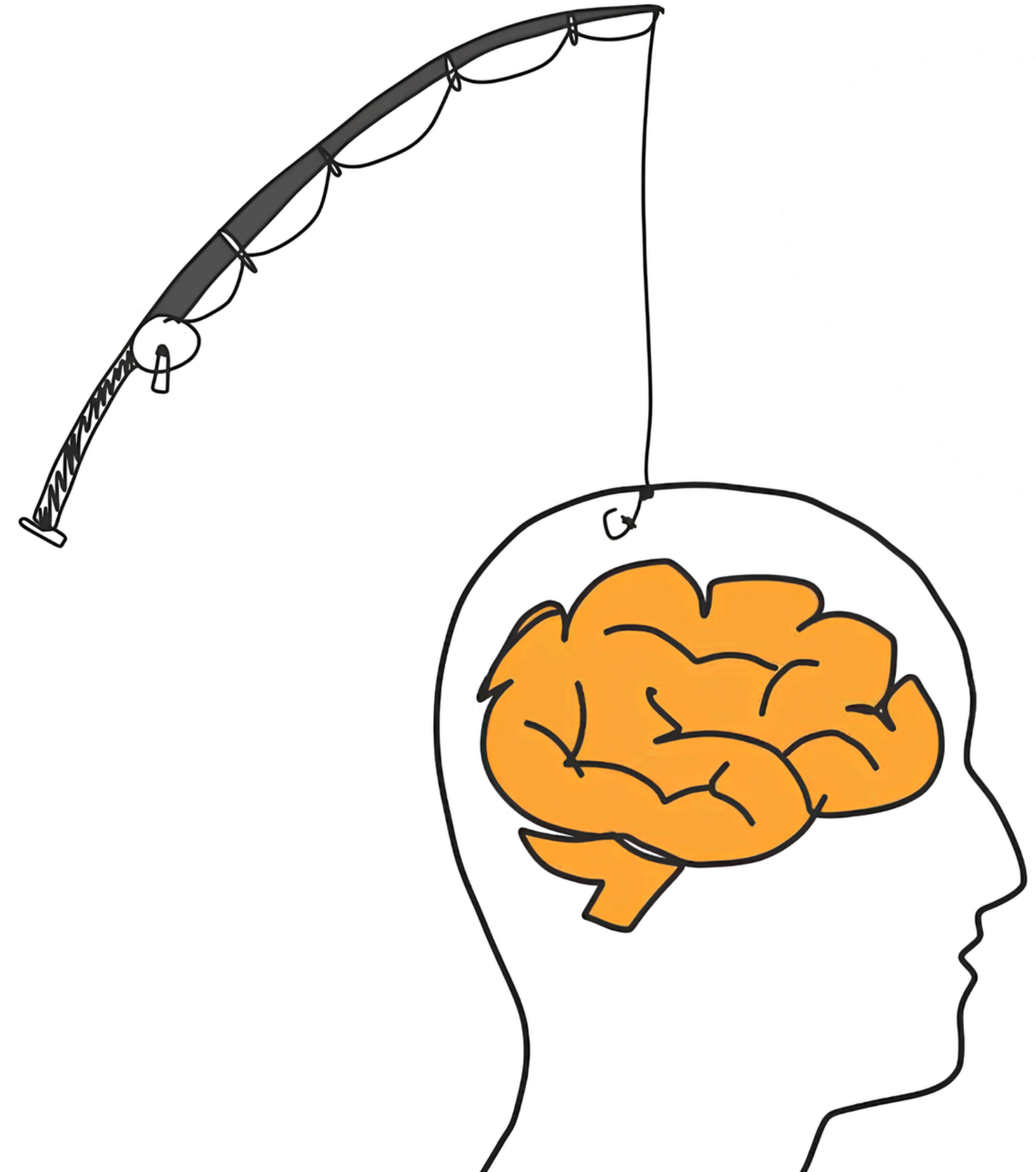
SECTION

Retrieval Strategies

Dense search has real blind spots — here's what fills them



Session 16 of 40



TWO GENUINELY DIFFERENT WAYS TO MATCH

Dense Retrieval vs. Sparse Retrieval (BM25)

Worked example — real BM25, no network needed at all



```
from rank_bm25 import BM25Okapi

tokenized_docs = [doc.lower().split() for doc in documents]
bm25 = BM25Okapi(tokenized_docs)

scores = bm25.get_scores(query.lower().split())
# Ranks by exact keyword overlap -- "stocks" will NOT match "stock"
# (no stemming), unlike dense retrieval's semantic matching
```

A real finding worth remembering

Query "stock market today" against a document containing "Stocks fell three percent today":

BM25 scores it LOWER than expected — "stocks" (plural) does not match "stock" in the query. No stemming, no meaning, only exact tokens.

Dense retrieval (M6·S2, M8·S1) would catch this instantly — "stocks" and "stock" land close in embedding space. That's the exact trade-off this slide is about.

ASKING THE QUESTION MULTIPLE WAYS

Multi-Query & Query Expansion

Query expansion: generating alternate phrasings of a user's query, then retrieving with ALL of them and merging the results — covers cases where the user's exact wording doesn't match the documents' wording.

Worked example — using Groq to generate alternates

```
def expand_query(query, groq_client):  
    prompt = f"Generate 2 alternate phrasings of this search query: {query}"  
    response = groq_client.chat.completions.create(  
        model="openai/gpt-oss-20b",  
        messages=[{"role": "user", "content": prompt}],  
    )  
    alt_queries = parse_alternatives(response.choices[0].message.content)  
    return [query] + alt_queries  # search with ALL of them, merge results
```

Real trade-off

Multi-query means multiple embedding calls AND multiple retrieval calls per user question — real added latency and cost (M4-S1 callback), traded for real improved recall on oddly-worded queries.

Worth it when queries are genuinely varied (real users, real phrasing); overkill for a narrow, well-controlled query interface.

Parent-Child Document Retrieval

Parent-child retrieval: search using SMALL chunks (precise matching), but return their LARGER parent chunk for full context — directly answering Slide 6's "too small loses context" problem.

Worked example

```
# Small chunks for PRECISE matching, but return the larger PARENT
# chunk they belong to for full context -- best of both sizes.
parent_map = {} # child_chunk_id -> parent_chunk_text

for parent in parent_chunks:
    children = small_splitter.split_text(parent)
    for child in children:
        parent_map[child] = parent # remember which parent this child came from

# At query time: search small chunks (precise), return their parents (context)
best_child = dense_search(query, list(parent_map.keys()))[0]
context = parent_map[best_child] # the bigger, more complete chunk
```

Why build this manually

LangChain does have a built-in version — but it recently moved to a separate, actively-restructuring package. A plain dictionary mapping child chunks to parent text is a few lines, has zero fragile dependencies, and teaches the actual mechanism directly.

Same philosophy as M8-S1's LangChain introduction: understand the real mechanism first.



Break

15 minutes — then you build and compare chunking and retrieval strategies for real

4

SECTION

Hands-on Lab

Build real chunking pipelines and compare dense, sparse, and parent-child retrieval



Session 16 of 40

Today's Lab: Chunk It, Search It, Compare It



Part 1 — Build and compare chunking strategies

Real recursive splitting at 2+ chunk sizes; inspect real chunk boundaries

10 min



Part 2 — Build a real BM25 retriever

Real sparse keyword search — no network needed, fully verifiable

10 min



Part 3 — Compare dense vs. sparse, honestly

Same queries, both retrievers — find where they agree and where they genuinely differ

15 min



Part 4 — Build parent-child retrieval

Small chunks for search, real parent chunks returned for context

10 min

Parts 1, 2, and 4 need nothing but local Python — no key, no network. Part 3's dense side reuses M8-S1's embedding model.

5

SECTION

Discussion

Chunk size, context, and precision — the real trade-off, and how to choose



Session 16 of 40

THE TRIANGLE YOU CAN'T FULLY ESCAPE

Trade-offs: Chunk Size vs. Context vs. Precision

Larger chunks

More context per chunk, fewer total chunks to search — but less precise matching, more irrelevant text riding along

Smaller chunks

Sharper, more precise matching — but less standalone context, and more chunks to store and search

Parent-child (Slide 16)

Genuinely gets both — but adds real implementation complexity and a second data structure to keep correct

A REAL STARTING POINT, NOT A FIXED RULE

Strategy Selection Guide

Exact terms matter (codes, names, IDs)

Sparse (BM25) or hybrid — dense retrieval alone will miss exact-token matches (Slide 14's finding)

Meaning matters more than exact wording

Dense retrieval — semantic similarity is the whole point

Users phrase things very differently

Add multi-query expansion — worth the extra real cost when input variety is genuinely high

Chunks lose too much context alone

Parent-child retrieval — precise search, full context on return

Not sure yet

Start with dense + recursive splitting (this session's defaults) — add strategies only once you've measured a real gap (Session 3)

BEFORE WE MOVE ON

Common Misconceptions



"Smaller chunks are always more precise, so always use them."

Precision without context can produce technically-relevant but practically useless retrieval (Slide 6's "\$50 for WHAT" problem).



"BM25 is an outdated technique, dense retrieval replaced it."

Sparse retrieval remains genuinely stronger on exact-term matching — many real production systems still combine both.



"Recursive splitting is the same as semantic chunking."

It's a smarter FIXED-SIZE approach that prefers clean boundaries — true semantic chunking (splitting by meaning shifts) is a different, more advanced technique.



"More retrieval strategies is always a stronger pipeline."

Each added strategy is real, added complexity and cost (Slide 15's multi-query trade-off) — justify each one with a measured gap, not by default.

SESSION SUMMARY

Key Takeaways

1

Chunking is a real design decision, not a preprocessing afterthought — size and boundaries quietly cap what retrieval can ever find

2

Recursive splitting is the practical default: prefers clean boundaries, but prioritizes a target size

3

Dense and sparse (BM25) retrieval fail in genuinely different, real ways — exact terms need sparse, meaning needs dense

4

Parent-child retrieval, multi-query expansion, and every added strategy are real trade-offs — add them for a measured reason, not by default

BEFORE NEXT SESSION

Assignment & What's Next



Assignment: Chunking & Retrieval Comparison

- Compare 2+ chunk sizes and 2+ retrieval strategies
- Test with queries designed to expose real differences
- Deliver a strategy recommendation with evidence

UP NEXT

Module 8, Session 3: Re-ranking & Evaluation

Real metrics (Recall@K, MRR, NDCG), cross-encoder re-ranking, and hybrid search — finally measuring, precisely, what today's comparisons only eyeballed.

Questions? Let's talk.